

Algorithm for file updates in Python

Project description

This project simulates a real world cybersecurity scenario, where I would work at a health care company, and part of my job is to update a file that identifies the employees who can access restricted content. I worked with two objects, the `allow_list.txt` file, which contains IP addresses permitted to sign into the restricted subnetwork, and the `remove_list`, which contains the IP addresses that can not sign into the subnetwork. My task was to create a Python algorithm to open the allow list file, read it, and compare it to the remove list, in order to identify and remove the prohibited IP address.

Open the file that contains the allow list

First, the `allow_list.txt` content was stored in the `import_file` variable. Next, the `remove_list` containing the blocked IP addresses. And finally, the `import_file` was opened using the `with` statement, and the `.open()` method followed by two parameters, the `import_file` variable, and the `"r"` command, this second parameter is a command that orders the `with` to read.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted inform
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

Figure 1: Opening the `import_file` variable

Running this code will produce an error because it only contains the first line of the `with` statement.

Read the file contents

To read the contents, I converted the `allow_list.txt` file into a string and used the `.read()` method. Then, I stored the file content in the `ip_addresses` variable and printed it out with the built-in Python function `print()`.

```
with open(import_file, "r") as file:
```

```
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
```

```
    ip_addresses = file.read()
```

```
# Display `ip_addresses`
```

```
print(ip_addresses)
```

```
ip_address
192.168.25.60
192.168.205.12
192.168.97.225
192.168.6.9
192.168.52.90
192.168.158.170
192.168.90.124
192.168.186.176
192.168.133.188
192.168.203.198
192.168.201.40
192.168.218.219
192.168.52.37
192.168.156.224
192.168.60.153
192.168.58.57
192.168.69.116
```

Figure 2: Read function to display IP addresses

Convert the string into a list

In order to remove prohibited IP addresses from the `ip_addresses`, the same needs to be in a list format. I did this by using the `.split()` method.

```
with open(import_file, "r") as file:
```

```
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
```

```
    ip_addresses = file.read()
```

```
# Use `.split()` to convert `ip_addresses` from a string to a list
```

```
ip_addresses = ip_addresses.split()
```

```
# Display `ip_addresses`
```

```
print(ip_addresses)
```

```
['ip_address', '192.168.25.60', '192.168.205.12', '192.168.97.225', '192.168.6.9', '192.168.52.90',  
'192.168.158.170', '192.168.90.124', '192.168.186.176', '192.168.133.188', '192.168.203.198', '192.16  
8.201.40', '192.168.218.219', '192.168.52.37', '192.168.156.224', '192.168.60.153', '192.168.58.57',  
'192.168.69.116']
```

Figure 3: Split function to convert ip addresses to a list format.

Iterate through the remove list

In order to use iterative statements, the content needs to be in a list format. Now that the `ip_addresses` variable is in list format, I can use the iterative statement `for` loop that will further iterate through the `remove_list`. The variable `element` was used as the loop variable.

```
for element in ip_addresses:
    # Display `element` in every iteration
    print(element)
```

```
ip_address
192.168.205.12
192.168.6.9
192.168.52.90
192.168.90.124
192.168.186.176
192.168.133.188
192.168.218.219
192.168.52.37
192.168.156.224
192.168.60.153
192.168.69.116
```

Figure 4: For loop to iterate and print each IP address

Remove IP addresses that are on the remove list

In the body of the iterative statement, I implemented the code that will remove all the IP addresses from the allow list that are also on the remove list. I used the loop variable `element` that will iterate through the `ip_addresses` list, each iteration will take and compare if the same is on `remove_list`, if so, the `.remove()` method will remove it from the list.

```
for element in ip_addresses:
    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:
        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)
# Display `ip_addresses`
print(ip_addresses)
```

```
['ip_address', '192.168.25.60', '192.168.205.12', '192.168.6.9', '192.168.52.90', '192.168.90.124',
'192.168.186.176', '192.168.133.188', '192.168.203.198', '192.168.218.219', '192.168.52.37', '192.16
8.156.224', '192.168.60.153', '192.168.69.116']
```

Figure 5: Removing the prohibited ID addresses from the list

Update the file with the revised list of IP addresses

Now that the prohibited IP addresses were removed, I completed the algorithm by updating the file with the revised list. This was done by converting `ip_addresses` back into a string using the `.join()` method. Then, the `with` was used again, but now, with the `.write()` method to write over the file assigned to the `import_file` variable.

```
# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file
|
with open(import_file, "w") as file:
    # Rewrite the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)
```

Figure 6: Saving the updated list

Summary

This scenario covered a lot of important functions and methods in Python. Knowing how to import files, convert, read and write are useful skills for the cyber security career, especially when dealing with log review tasks.